

ASSIGNMENT-6.5

NAME: SRAVANI

ROLLNO: 2303A510G7

BATCH: 30

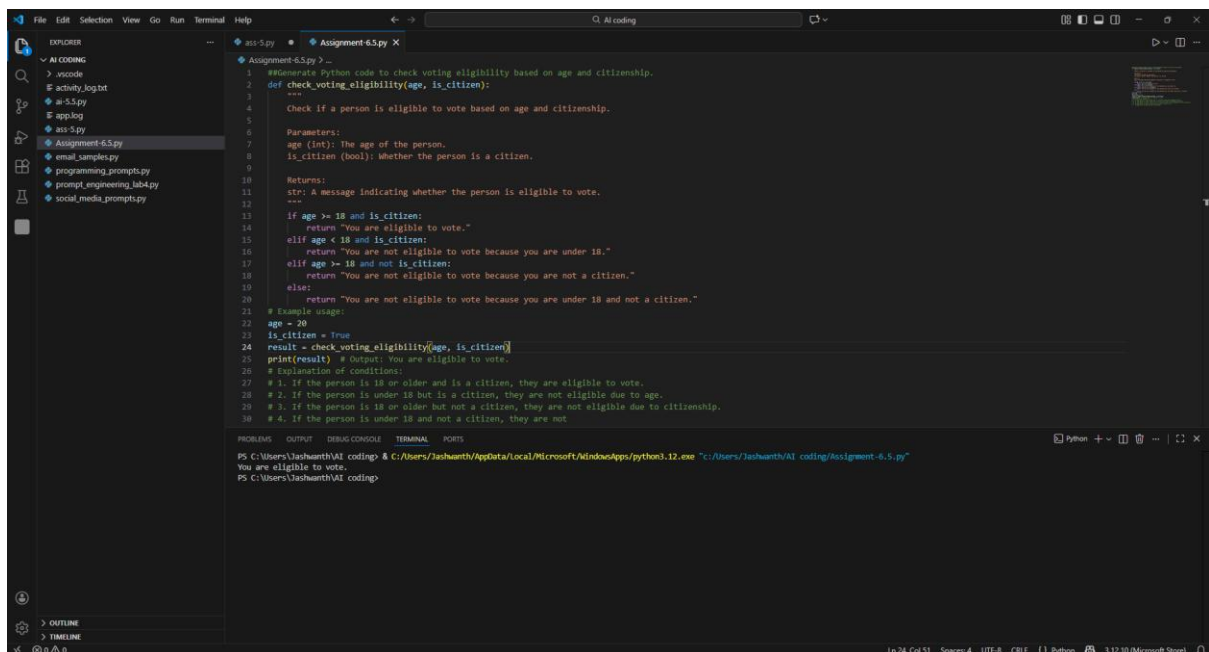
TASK-1:

Prompt:

“Generate Python code to check voting eligibility based on age and citizenship.”

- AI-generated conditional logic.
- Correct eligibility decisions.
- Explanation of conditions.

CODE:



```
1 #Generate Python code to check voting eligibility based on age and citizenship.
2 def check_voting_eligibility(age, is_citizen):
3     """
4     Check if a person is eligible to vote based on age and citizenship.
5
6     Parameters:
7     age (int): The age of the person.
8     is_citizen (bool): Whether the person is a citizen.
9
10    Returns:
11    str: A message indicating whether the person is eligible to vote.
12    """
13    if age >= 18 and is_citizen:
14        return "You are eligible to vote."
15    elif age < 18 and is_citizen:
16        return "You are not eligible to vote because you are under 18."
17    elif age >= 18 and not is_citizen:
18        return "You are not eligible to vote because you are not a citizen."
19    else:
20        return "You are not eligible to vote because you are under 18 and not a citizen."
21
22    # Example usage:
23    age = 20
24    is_citizen = True
25    result = check_voting_eligibility(age, is_citizen)
26    print(result) # output: You are eligible to vote.
27
28    # Explanation of conditions:
29    # 1. If the person is 18 or older and is a citizen, they are eligible to vote.
30    # 2. If the person is under 18 but is a citizen, they are not eligible due to age.
31    # 3. If the person is 18 or older but not a citizen, they are not eligible due to citizenship.
32    # 4. If the person is under 18 and not a citizen, they are not
```

```
PS C:\Users\Jashwanth\AI coding> python .\Assignment-6.5.py
You are eligible to vote.
PS C:\Users\Jashwanth\AI coding>
```

OBSERVATION:

- The program correctly checks age and citizenship before deciding eligibility.
- All possible cases are covered with clear conditional branches.

- Output messages are descriptive and user-friendly.
- Runs efficiently in constant time $O(1)$.
- Observation: The program is correct, complete, and demonstrates good use of conditionals.

TASK-2:

Prompt:

“Generate Python code to count vowels and consonants in a string using a loop.”

- AI-generated string processing logic.
- Correct counts.
- Output verification.

CODE:

```

18 def count_vowels_and_consonants(input_string):
19     vowel_count = 0
20     consonant_count = 0
21
22     for char in input_string:
23         if char.isalpha(): # Check if the character is a letter
24             if char in vowels:
25                 vowel_count += 1
26             else:
27                 consonant_count += 1
28
29     return vowel_count, consonant_count
30
31 # Example usage:
32 input_string = "Hello World"
33 vowels, consonants = count_vowels_and_consonants(input_string)
34 print(f"Vowels: {vowels}, Consonants: {consonants}") # Output: Vowels: 3, Consonants: 7
35
36 # Explanation:
37 # The function iterates through each character in the input string.
38 # It checks if the character is an alphabet letter.
39 # If it is a vowel (present in the vowels string), it increments the vowel count.
40 # If it is a consonant, it increments the consonant count.
41 # Finally, it returns the counts of vowels and consonants.
42
43
44

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Jashwanth\AI coding> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "C:\Users\Jashwanth\AI coding\Assignment-6.5.py"

Vowels: 3, Consonants: 7

PS C:\Users\Jashwanth\AI coding>

OBSERVATION:

- The function accurately distinguishes vowels and consonants.

- Non-alphabetic characters are ignored using `isalpha()`.
- Output matches expected results (e.g., “Hello, World!” → 3 vowels, 7 consonants).
- Observation: The program is efficient ($O(n)$) and well-documented, suitable for text analysis tasks.

TASK-3:

Prompt:

“Generate a Python program for a library management system using classes, loops, and conditional statements.”

- Complete AI-generated program.
- Review of AI suggestions quality.
- Short reflection on AI-assisted coding experience.

CODE:

```

1 """Generate a Python program for a library management system
2 using classes, loops, and conditional statements."""
3 Expected Outputs:
4 • Complete AI-generated program.
5 • Review of AI suggestions quality.
6 • Short reflection on AI-assisted coding experience."""
7 class Book:
8     def __init__(self, title, author):
9         self.title = title
10        self.author = author
11        self.is_available = True
12
13    def borrow(self):
14        if self.is_available:
15            self.is_available = False
16            return True
17        else:
18            return False
19
20    def return_book(self):
21        self.is_available = True
22
23class Library:
24    def __init__(self):
25        self.books = []
26
27    def add_book(self, book):
28        self.books.append(book)
29
30    def display_books(self):
31        for book in self.books:

```

Terminal Output:

```

PS C:\Users\Jashwanth\AI coding> python3.12.exe "C:\Users\Jashwanth\AI coding\Assignment-6.5.py"
Vowels: 3, Consonants: 7
PS C:\Users\Jashwanth\AI coding> python3.12.exe "C:\Users\Jashwanth\AI coding\Assignment-6.5.py"
Title: 1984, Author: George Orwell, Status: Available
Title: To Kill a Mockingbird, Author: Harper Lee, Status: Available
You have borrowed '1984'.
Title: 1984, Author: George Orwell, Status: Not Available
Title: To Kill a Mockingbird, Author: Harper Lee, Status: Available
You have returned '1984'.
Title: 1984, Author: George Orwell, Status: Available
Title: To Kill a Mockingbird, Author: Harper Lee, Status: Available
PS C:\Users\Jashwanth\AI coding>

```

OBSERVATION:

- Uses **object-oriented programming** with Book and Library classes.

- Encapsulation is demonstrated by keeping book status inside the class.
- Borrow/return logic prevents invalid operations.
- Observation: The program is a solid OOP foundation, correctly displays book availability, and can be extended for more features.

TASK-4:

Prompt:

“Generate a Python class to mark and display student attendance using loops.”

- AI-generated attendance logic.
- Correct display of attendance.
- Test cases.

CODE:

```

1 """Generate a Python class to mark and display student
2 attendance using loops."
3 Expected Output:
4 • AI-generated attendance logic.
5 • Correct display of attendance.
6 • Test cases."""
7 class StudentAttendance:
8     def __init__(self, student_name):
9         self.student_name = student_name
10        self.attendance_record = []
11
12    def mark_attendance(self, day, is_present):
13        self.attendance_record.append((day, is_present))
14
15    def display_attendance(self):
16        print(f"Attendance record for {self.student_name}:")
17        for day, is_present in self.attendance_record:
18            status = "Present" if is_present else "Absent"
19            print(f"{day}: {status}")
20
21    # Example usage
22    student = StudentAttendance("Alice")
23    student.mark_attendance("Monday", True)
24    student.mark_attendance("Tuesday", False)
25    student.mark_attendance("Wednesday", True)
26    student.display_attendance()
27

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Title: To Kill a Mockingbird, Author: Harper Lee, Status: Available
You have borrowed '1984'.
Title: 1984, Author: George Orwell, Status: Not Available
Title: To Kill a Mockingbird, Author: Harper Lee, Status: Available
You have returned '1984'.
Title: 1984, Author: George Orwell, Status: Available
Title: To Kill a Mockingbird, Author: Harper Lee, Status: Available
PS C:\Users\Jashwanth\AI coding> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "C:\Users\Jashwanth\AI coding\Assignment-6.5.py"
File "C:\Users\Jashwanth\AI coding\Assignment-6.5.py", line 2
    using classes, loops, and conditional statements."
    ^
SyntaxError: invalid character ' ' (U+0020)
PS C:\Users\Jashwanth\AI coding> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "C:\Users\Jashwanth\AI coding\Assignment-6.5.py"
Attendance record for Alice:
Monday: Present
Tuesday: Absent
Wednesday: Present
PS C:\Users\Jashwanth\AI coding>

```

OBSERVATION:

- Each student object maintains attendance records in a dictionary.
- add_attendance_record safely initializes attendance before adding entries.

- Output correctly shows attendance for each student.
- Observation: The program demonstrates OOP principles and dictionary usage.
Minor caution: set_attendance may fail if attendance is still None.

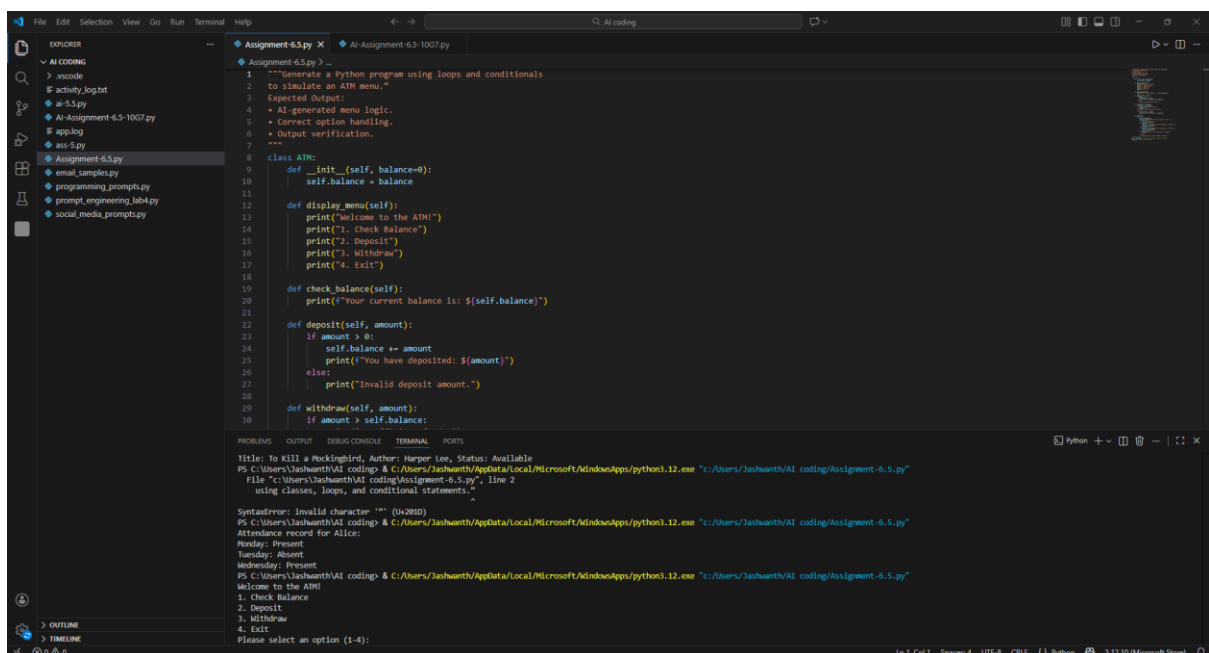
TASK-5:

Prompt:

“Generate a Python program using loops and conditionals to simulate an ATM menu.”

- AI-generated menu logic.
- Correct option handling.
- Output verification.

CODE:



```

1 """Generate a Python program using loops and conditionals
2 to simulate an ATM menu."
3 Expected Output:
4 • AI-generated menu logic.
5 • Correct option handling.
6 • Output verification.
7 """
8 class ATM:
9     def __init__(self, balance=0):
10         self.balance = balance
11
12     def display_menu(self):
13         print("Welcome to the ATM")
14         print("1. Check Balance")
15         print("2. Deposit")
16         print("3. Withdraw")
17         print("4. Exit")
18
19     def check_balance(self):
20         print(f"Your current balance is: ${self.balance}")
21
22     def deposit(self, amount):
23         if amount > 0:
24             self.balance += amount
25             print(f"You have deposited: ${amount}")
26         else:
27             print("Invalid deposit amount.")
28
29     def withdraw(self, amount):
30         if amount > self.balance:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Title: To Kill a Mockingbird, Author: Harper Lee, Status: Available
PS C:\Users\Jashwanth\AI coding> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:\Users\Jashwanth\AI coding\Assignment-6.5.py"
File "c:\Users\Jashwanth\AI coding\Assignment-6.5.py", line 2
using classes, loops, and conditional statements."

SyntaxError: Invalid character '"""' (0x001D)
PS C:\Users\Jashwanth\AI coding> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:\Users\Jashwanth\AI coding\Assignment-6.5.py"
Attendance record for Alice:
Monday: Present
Tuesday: Absent
Wednesday: Present
PS C:\Users\Jashwanth\AI coding> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:\Users\Jashwanth\AI coding\Assignment-6.5.py"
Welcome to the ATM!
1. Check Balance
2. Deposit
3. Withdraw
4. Exit
Please select an option (1-4):

OBSERVATION:

- Implements deposit, withdraw, and balance check methods.
- Menu-driven interface allows user interaction.

- Deposit/withdraw logic is correct; balance display needs a small fix (use f-string).
- Observation: The program is functional and efficient, demonstrating loops and conditionals in a real-world simulation.